Arquitetura YallaFinance — visão ponta a ponta

Data de referência: 02/11/2026.

Este documento descreve como o sistema funciona de ponta a ponta: do usuário no frontend até o banco e os provedores financeiros. Para subir o ambiente, consulte o README.

1. Componentes

Componente	Tecnologia	Responsabilidade
Frontend	Next.js 16, React 19, MUI	Telas operacionais e checkout público. Não fala com a API diretamente.
BFF do Next	Rotas do Next	Lê a sessão no cookie HttpOnly e repassa as chamadas à API com o token no servidor.
WebApi	ASP.NET Core	Controllers finos, autenticação por sessão e autorização por funcionalidade.
Application	MediatR, FluentValidation	Casos de uso, validação e contratos (repositórios e serviços). Não conhece fornecedores.
Infrastructure	EF Core, Npgsql, Refit	Repositórios PostgreSQL, providers (Nibo, Asaas, Sicoob) e serviços de pagamento.
Worker	Processo separado	Dois serviços em segundo plano: execução de Jobs (com retry automático) e sincronização de recebimentos Pix.
Domain	Tipos de domínio	Enums e objetos de valor, sem dependências.
PostgreSQL	Container Docker	Esquemas `core` (técnico e acesso) e `finance` (financeiro). É o único componente em container no ambiente local.

Dependências entre camadas: WebApi e Worker usam Application; Infrastructure implementa os contratos de Application. Application não referencia Infrastructure.

2. Autenticação e autorização

- O login cria uma sessão em `core.user_session`. O token fica em cookie HttpOnly (`yalla-finance-session`) no navegador e não chega ao JavaScript.
- O servidor do Next consulta a API para validar a sessão em cada leitura. Não há cache de sessão no Next.
- Os endpoints da API são protegidos por funcionalidade (`RequireFunctionality`), vinculada aos perfis de acesso do usuário.
- O tenant vem sempre do contexto autenticado, nunca do corpo da requisição.
- Integrações externas usam um Bearer de integração separado das sessões. Sua chave é gerada aleatoriamente a cada inicialização da API.

3. Configurações

Configurações da empresa ficam em `core.tenant_setting`, com chave única por tenant, grupo e chave. A tela de configurações tem as abas:

- **Empresa:** parâmetros gerais da empresa.

- **Conexões:** credenciais e parâmetros dos providers, em `core.external_connection` e tabelas relacionadas. Cada conexão de Sicoob e Asaas mostra o campo **Período de consulta (dias)** para recebimentos Pix.
- **Jobs:** retentativas automáticas por tipo de Job (grupo `job_retry_limit`, de 0 a 20). Zero ou campo vazio desativa.

Conexão com o banco e demais parâmetros de infraestrutura são definidos por variáveis de ambiente no processo (API, Worker e servidor Next), conforme o README.

4. Fluxo de cobrança

1. O operador cria a cobrança pelo frontend. A chamada passa pelo BFF e chega à API.
2. O serviço de criação de cobrança grava a cobrança, os itens e os vínculos em uma transação, com chave de idempotência. A persistência é confirmada antes de agendar o próximo passo.
3. Depois do commit, a aplicação cria o Job de criação da cobrança no provider financeiro (Nibo).
4. O worker executa o Job, que cria a cobrança externa. Cada item recebe o resultado.
5. Resultados de integração: `Completed`, `Partial`, `Indeterminate` ou `Rejected`. Em `Partial`, `Indeterminate` e `Rejected` o Job termina em `Failed` com diagnóstico.
6. Se a integração falhar, o reprocessamento manual está sempre disponível, e o retry automático segue a configuração da aba Jobs.

5. Fluxo de pagamento (checkout público)

1. Uma solicitação de pagamento é criada e gera um link público. O link usa apenas o hash do token.
2. O cliente abre o checkout e escolhe o meio de pagamento. Quando há um único método disponível, ele já vem selecionado.
3. Uma tentativa de pagamento é aberta com chave de idempotência.
4. **Pix:** o provider cria a cobrança com o identificador (`txid`) da tentativa.
   - Antes de criar, o provider consulta a cobrança pela referência. Se ela já existe e confere com o pedido, reaproveita o QR Code. Se não existe, cria. Se a consulta falha, devolve erro sem criar.
   - Resposta que contradiz o pedido (pagador, descrição, valor, expiração) vira erro para o cliente.
   - O prazo máximo da criação é de **15 segundos**. Estourado o prazo, o cliente recebe erro com a opção de tentar novamente.
5. **Cartão e boleto:** seguem o mesmo ciclo, com os serviços próprios de cada meio.
6. O cliente recebe o QR Code ou os dados de pagamento, ou um erro com "Você pode tentar novamente".
7. Estados de uma tentativa: `Pending`, `Processing`, `AwaitingPayment`, `Succeeded`, `Failed`, `Expired`, `Canceled` e `Unknown`.

6. Recebimento e baixa (sincronização)

Quem executa é o serviço de sincronização do Worker, em ciclos de 15 segundos.

1. Para cada conexão ativa, a rotina calcula a janela padrão: os últimos **7 dias** (configurável por conexão, de 1 a 31).
2. Se o provider exige que a consulta fique dentro do mesmo mês civil (o Sicoob exige), a janela é dividida em segmentos mensais, no fuso de Brasília.
3. Cada segmento é lido página a página. Itens com dado inválido são pulados e registrados em log, e a página continua com os demais.
4. Os recebimentos são associados à tentativa pelo `txid`.
5. Um recebimento confirmado confirma a tentativa e dispara a baixa da cobrança.

Se a consulta falha, a rotina registra o aviso e tenta de novo no próximo ciclo. Como a janela é móvel, perdas além de 7 dias exigem ajuste manual.

7. Jobs

- Um Job tem tipo, chave, payload e status: `Pending`, `Processing`, `Success` ou `Failed`.

- O serviço de Jobs do Worker consulta a fila a cada 5 segundos (`Jobs:PollingIntervalSeconds`) e pega até 10 Jobs por ciclo (`Jobs:BatchSize`). A reserva usa bloqueio (`FOR UPDATE SKIP LOCKED`), então vários Workers podem rodar ao mesmo tempo sem processar o mesmo Job.
- A reserva marca o Job como `Processing` e abre uma execução em `core.job_execution` .
- O handler executa o evento desde o início e confere o estado atual antes de agir, então uma nova tentativa não duplica efeito.
- Ao final, o Job vira `Success` ou `Failed` , com diagnóstico.
- **Retry automático:** a cada ciclo, um varredor reenfileira Jobs em `Failed` cujo tipo esteja configurado em `job_retry_limit` , dentro do limite e após o intervalo de espera (30 s dobrando a cada tentativa, teto de 15 min, com variação de até 20% por Job). Sem configuração, nenhum Job é retentado.
- **Reprocessamento manual:** continua disponível pela tela de Jobs, independentemente da configuração.

8. Banco de dados

- Esquema `core` : tenants, usuários, sessões, perfis e funcionalidades, conexões externas, configurações, Jobs e execuções.
- Esquema `finance` : clientes, contratos, categorias, formas de pagamento, débitos, cobranças e itens, solicitações e tentativas de pagamento, transações e recibos.
- Triggers protegem o histórico de pagamento: a identidade de tentativas e solicitações não pode ser alterada, os recibos de comando são somente de inserção e os métodos da solicitação não são reescritos.
- Tabelas auditadas registram quem criou e quem alterou, com o usuário autenticado.
- O ambiente de referência é restaurado a partir de um backup fornecido pela equipe, que já contém os dados. Os scripts de migração não fazem parte do repositório atual.

9. Operação

- O PostgreSQL sobe com Docker Compose, com volume persistente. API, Worker e frontend rodam como processos.
- Logs: hoje são gravados no console da API e do Worker. Não há arquivo persistente.
- Os passos de subida de cada parte estão no README.

10. Telas do frontend

- **Acesso:** login.
- **Financeiro:** início, débitos (novo e detalhe), cobranças (nova, revisão, confirmação e sucesso), cobrança direta, solicitações de pagamento e resultado.
- **Cadastros:** perfis de acesso, usuários e sessões ativas.
- **Operação:** Jobs (lista e detalhe) e configurações (abas Empresa, Conexões e Jobs).
- **Público:** checkout de pagamento em `/pay/[token]` , sem login.

11. Pendências conhecidas

Tema	Situação
Restauração do backup	Validar, após a restauração, que a API e o Worker sobem e que o login funciona.
Conta administrativa da carga base	`admin@yalla.com.br` é criada pela carga base. Senha já trocada no backup. Conta mantida para testes; decidir se será desativada antes de produção.
Chave do Bearer de integração	Gerada a cada start da API. Reiniciar invalida os tokens de integração. A chave do `appsettings` não é usada. Definir mecanismo de assinatura e rotação para produção.
Encaminhamento de proxy	A API não configura `UseForwardedHeaders` . Validar o caminho HTTPS antes de publicar atrás de proxy.
CORS	Não configurar em produção: o navegador não chama a API. Lista fixa vale só para desenvolvimento.

Tema	Situação
Dados mock	Movimentações de frota são lidas de JSON em memória. Os demais dados mock são reservados para débito e cobrança avulsos (outra fase).
Logs persistentes	Só console. Falta gravação em arquivo por dia.
Ponto de retomada da sincronização	Decidido não implementar. Janela móvel de 7 dias, com ajuste manual.
Alerta para falhas repetidas	Adiado para uma fase futura de alertas operacionais.
Confirmação com o Sicoob	Comportamento ao repetir um `txid` não documentado. Risco aceito, a confirmar com o banco.
Payload de recebimentos	Hoje guarda o retorno completo do provider, inclusive dados do pagador. Redução adiada.
Origem do frontend na API	`frontend/.env.example` aponta `API_URL` para a porta 5000, e o perfil de desenvolvimento da API usa 5077. Alinhar antes de subir.